

RoK4 Reward Structure

작성일: 2026-07-15 최종 업데이트: 2026-09-18

이 문서는 RoK4-Isaac-Velocity-Flat-v0 task의 현재 reward 구조와 reward function 설정을 정리한다. 현재 reward는 Isaac Lab G1 velocity task 구조를 출발점으로 RoK4 ADAPT actuator 좌표, direct velocity command, standing transition에 맞게 조정한 flat walking 실험이다. 현재 개발 baseline은 아래의 2026-09-17 착지 직전 모서리 비용 강화 설정으로 학습한 다음 정책이다. Run: 2026-09-17_11-46-47_concurrent_estimator_edgevel01_pre5_window100_tdpitch1_fresh25k, checkpoint: model_24999.pt, post-training code snapshot: 903318c. 9월 18일 사용자가 Isaac Sim keyboard Teleop에서 착지가 부드러워졌다고 관찰했다. Sim2Sim/Sim2Real 검증은 대기 상태이며 전체 baseline 목록은 README 상단 표를 기준으로 한다. 다음 reference는 과거 symmetry 비교용으로 보존한 정책이다: 2026-07-24_19-34-26_symmetry_aug_nojumps2_swing_roll100_fresh/model_9999.pt 이며 experimental Yunho Symmetry ADAPT v1 으로 기록한다. 이 checkpoint는 domain randomization과 policy observation noise를 조정하기 전의 비교 기준이며, 이후 실험에서도 보존한다.

아래 경로에서 RoK4: 는 \${ROK4_LAB_ROOT} 를, Isaac Lab: 은 \${ISAACLAB_ROOT} 를 기준으로 한 상대경로를 뜻한다. 두 변수는 각 사용자가 clone한 저장소 root를 가리킨다.

```
export ROK4_LAB_ROOT="${HOME}/rok4_lab"
export ISAACLAB_ROOT="${HOME}/IsaacLab"
```

Observation noise, reset state randomization, physics DR의 상세 범위와 적용 주기는 docs/rok4_randomization_and_noise_ko.rst 를 기준 문서로 사용한다. 자동 velocity/force 외란의 좌표계, impulse와 recovery 검증은 docs/rok4_disturbance_and_recovery_ko.rst 를 기준 문서로 사용한다.

관련 파일

역할	파일
RoK4 flat task 설정	RoK4: .../config/rok4/flat_env_cfg.py
RoK4 ADAPT transmission/actuator	RoK4: .../assets/robots/rok4_adapt.py
RoK4 actuator action/observation	RoK4: .../velocity/mdp/actions.py, observations.py
RoK4 actuator reward 함수	RoK4: .../velocity/mdp/rewards.py
RoK4 domain randomization 설정	RoK4: .../config/rok4/domain_randomization_cfg.py
공통 locomotion reward term 기본값	Isaac Lab: .../velocity/velocity_env_cfg.py
locomotion 전용 reward 함수	Isaac Lab: .../velocity/mdp/rewards.py
Isaac Lab 공통 reward 함수	Isaac Lab: isaacsim/envs/mdp/rewards.py
mdp namespace 연결	Isaac Lab: .../velocity/mdp/__init__.py

Actuator interface와 reward 연결

현재 reward는 단순히 joint 값을 actuator라고 이름만 바꾼 것이 아니다. action target, 실제 상태, PD torque가 각각 ADAPT 행렬을 통과하며, reward는 그 runtime actuator 값 또는 mapped joint target을 명시적으로 사용한다.

```
rok4.py
└─ 링크 길이, actuator gain/action scale/limit 설정
  └─ rok4_adapt.py
    └─ RoK4AdaptTransmission:  $J/J^{-1}/J^T/J^{-T}$ 
      └─ RoK4AdaptActuator: actuator PD와  $\tau_{\psi}/\tau_q$  계산
        │
        └─ +-> mdp/actions.py
          │   raw action ->  $\psi_{\text{target}}$  ->  $q_{\text{target}}$ 
          │
          └─ +-> mdp/observations.py
            │    $q/\dot{q}$  ->  $\psi/\dot{\psi}$  observation
            │
            └─ +-> mdp/rewards.py
              │   └─  $\tau_{\psi}$ ,  $\dot{\psi}$ ,  $\ddot{\psi}$  penalty
              │   └─ actuator torque/velocity limit 초과 penalty
              │   └─ clipped raw actuator action-rate penalty
              └─  $q_{\text{target}}$  joint-position-limit penalty
                └─ -> flat_env_cfg.py의 RoK4RewardsCfg
                  func + weight + params를 RewardTerm으로 구성
```

rok4_adapt.py, actions.py, observations.py, rewards.py 는 모두 rok4_lab 안의 로컬 파일이며, Isaac Lab 원본 source 를 수정하지 않는다. Isaac Lab의 부모 class와 mdp 함수는 import/상속/re-export해서 사용한다.

구조 요약

RoK4FlatEnvCfg 는 환경 설정이고, RoK4RewardsCfg 는 reward term 묶음이다. Reward 구조는 두 가지 관계로 나누어 보면 될 것같린다.

첫 번째는 config class의 상속/포함 관계이다.

```

RoK4FlatEnvCfg
├── 상속: LocomotionVelocityRoughEnvCfg
│   └── 위치:
│       IsaacLab/.../locomotion/velocity/velocity_env_cfg.py
├── rewards: RoK4RewardsCfg()
│   └── RoK4RewardsCfg
│       ├── 상속: RewardsCfg
│       │   └── 위치:
│       │       IsaacLab/.../locomotion/velocity/velocity_env_cfg.py
│       ├── 부모 RewardsCfg에서 물려받은 reward terms
│       │   ├── lin_vel_z_l2
│       │   ├── ang_vel_xy_l2
│       │   ├── flat_orientation_l2
│       │   └── undesired_contacts
│       └── RoK4RewardsCfg에서 새로 정의/override한 reward terms
│           ├── termination_penalty
│           ├── track_lin_vel_xy_exp
│           ├── track_ang_vel_z_exp
│           ├── feet_air_time
│           ├── base_height_l2
│           ├── feet_clearance
│           ├── no_jumps
│           ├── feet_slide
│           ├── feet_touchdown_velocity
│           ├── feet_touchdown_edge_velocity (직전 비용 x5 + 접촉 후 100 ms)
│           ├── feet_contact_velocity (현재 None: 비활성)
│           ├── feet_contact_force (현재 None: 비활성)
│           ├── feet_touchdown_acc (현재 None: 비활성)
│           ├── feet_flat_orientation_l2 (현재 None: 비활성)
│           ├── feet_swing_roll_l2 (swing 중 roll 억제)
│           ├── feet_swing_pitch_l2 (swing 중 pitch를 약하게 억제)
│           ├── FeetTouchdownPitchL2 (첫 착지에서 pitch 오차 1회 평가)
│           ├── feet_stance_width_l2 (현재 None: 비활성)
│           ├── feet_lateral_separation_l2 (signed lateral anti-cross)
│           ├── stand_still_joint_deviation_l1 (현재 None: no-standing-pose ablation)
│           ├── feet_standing_contact (exact-zero 명령의 미접촉 발 수)
│           ├── dof_pos_limits
│           └── joint_action_target_pos_limits

```

- └ joint_deviation_hip
- └ joint_deviation_hip_pitch
- └ joint_deviation_torso
- └ actuator_acc_l2
- └ actuator_torques_l2
- └ actuator_vel_l2
- └ actuator_velocity_limits
- └ actuator_torque_limits
- └ action_rate_l2
- └ second_action_rate_l2

두 번째는 각 reward term이 실제 계산 함수를 참조하는 관계이다.

```

RoK4FlatEnvCfg
└ rewards = RoK4RewardsCfg()
  └ RoK4RewardsCfg extends RewardsCfg
    └ 각 reward term은 RewardTermCfg/RewTerm
      └ func=mdp.xxx
        └ RoK4 로컬 mdp 함수
            source/rok4_tasks/.../velocity/mdp/rewards.py
        └ Isaac Lab 공통 mdp 함수
            IsaacLab/source/isaacsim/isaacsim/envs/mdp/rewards.py
        └ locomotion velocity 전용 mdp 함수
            IsaacLab/source/isaacsim_tasks/.../velocity/mdp/rewards.py

```

즉 RoK4FlatEnvCfg -> RoK4RewardsCfg 는 상속이 아니라 포함/사용 관계이고, RoK4RewardsCfg -> RewardsCfg 는 상속 관계이다. mdp/rewards.py 는 부모 클래스가 아니라 실제 reward 계산 함수가 들어 있는 함수 모음이다. RoK4는 로컬 rok4_tasks...velocity.mdp 를 import하며, 이 로컬 mdp는 Isaac Lab의 기존 locomotion mdp를 다시 export하고 RoK4 전용 reward 함수만 추가한다.

실제로는 다음처럼 이해하면 된다.

```

RoK4RewardsCfg는 RewardsCfg를 상속받아서 기본 reward term들을 물려받는다.
RoK4에 필요한 reward는 mdp.xxx 함수를 직접 지정해 새로 정의하거나 override한다.
RewardTermCfg/RewTerm은 mdp.xxx 함수, weight, params를 하나의 reward term으로 묶는다.

```

파일 배치 기준은 다음과 같다.

```
flat_env_cfg.py
└─ 어떤 reward term을 쓸지, weight를 얼마로 둘지, 어떤 body/joint에 적용할지 결정

velocity/mdp/rewards.py
└─ reward raw value를 어떻게 계산할지 정의
```

따라서 joint_action_target_pos_limits, actuator_torques_l2, actuator_vel_l2, actuator_acc_l2, actuator_velocity_limits, actuator_torque_limits, action_rate_l2, second_action_rate_l2 처럼 RoK4 전용 weighting을 쓰는 계산식은 velocity/mdp/rewards.py 에 두고, weight를 얼마로 사용할지는 RoK4RewardsCfg 에 둔다. 이 term은 이전 action 두 개가 필요하므로 reset 직후 첫 두 policy step에서는 penalty를 0으로 둔다.

Action-rate reward의 action 기준

현재 RoK4Lab의 action_rate_l2 와 second_action_rate_l2 는 env.action_manager.action 을 기준으로 현재 action 과 history의 차분을 만든다. Reward 함수는 별도 clamp를 하지 않는다. 하지만 표준 RoK4 RSL-RL 경로에서는 clip_actions=1.0 wrapper가 ActionManager 이전에 정책 출력을 제한하므로 이 action buffer는 scale 적용 전 clipped raw policy action이다. Reward는 이 raw 차분의 weighted 제곱합을 계산한다. Action scale은 적용하지 않지만 Hip Pitch/Knee에 해당하는 index 2,3,8,9 의 제곱 오차에는 RoK4 전용 완화 weight 0.5 를 적용한다.

RoK4FlatPP0RunnerCfg 에서 clip_actions = 1.0 을 명시하므로 Isaac Lab train.py 와 play.py 를 통해 실행할 때는 RSLVecEnvWrapper 가 action을 먼저 [-1, 1] 로 clamp한다. 따라서 reward가 보는 action은 wrapper에서 [-1, 1] 로 잘린 raw actuator action이다.

ROBOTIS K1 Rev1의 reward 함수도 별도 clamp 없이 ActionManager.action 을 읽지만, K1 runner에는 clip_actions 설정이 없어 기본값 None 을 사용한다. 따라서 K1은 unclipped raw action 차분, RoK4 표준 학습은 wrapper-clipped raw action 차분이라는 차이가 있다.

K1-inspired no-standing-pose 가중치 비교

현재 실험은 K1 설정을 그대로 복제하지 않는다. RoK4의 policy 주기는 0.01 s 이고 K1은 0.02 s 이며, action 차원과 scale 도 다르다. 다음 표는 이 차이를 남긴 상태에서 사용하는 중간 강도 설정이다.

2026-09-07 후속 실험에서는 첫 no-standing-pose 학습의 hip yaw/roll weight -0.1 만 -0.2 로 바꾼다. 비교 run은 2026-09-04_14-27-20_concurrent_estimator_k1style_nostand_bh5_hip010_hippitch001_ar05_ar2_01_fresh25k 이며 Teleop에서 팔자 자세와 제자리 stepping이 관찰되었다. 이번 변경은 팔자 자세를 먼저 확인하기 위한 단일 가중치 실험이다. Standing term은 None 을 유지하며 아래 표의 다른 현재 값과 외란/환경 구성도 변경하지 않는다. 팔자 감소가 제자리 stepping 해결을 의미하지 않으며 홍보와 recovery step 제한 여부도 평가해야 한다. 이후 사용자는 Isaac Sim에서 팔자 자세 개선과 홍보/회전/회복 동작 유지를 확인했으나 정지 stepping은 남았다. 같은 seed/checkpoint의 30초 true-velocity 대체 진

단도 stepping을 없애지 못했다. 이 결과만으로 모든 초기 상태에서 estimator 영향이 없다고 단정하지 않는다. 원본 학습/추론/export는 그대로이며 2026-09-08 후속 실험에서는 별도 feet_standing_contact=-0.1 항을 추가했다. 그 run의 model_5000.pt 도 제자리 stepping이 남았으며, 7122까지의 접촉 비용 추세도 거의 평평했다. 이후 -0.2 학습의 2026-09-08_16-59-33_concurrent_estimator_hip020_standcontact020_fresh25k/model_24999.pt 에서는 사용자가 Isaac Sim의 제자리 stepping 해소와 보행 유지를 확인했다. 이 run의 Sim2Sim/Sim2Real 검증은 보고되지 않았다. 2026-09-09에는 이 설정을 유지하고 착지 속도 reward만 링크 원점 기준으로 바꾸는 tdlink 실험을 진행했다. 중간 로그에서 심한 조기 종료가 관찰되어, 사용자 요청으로 현재 코드를 발 COM 기준으로 복원했다. 복원 fresh run은 2026-09-10에 24,999 iteration까지 완료했다. 아래 soft-landing 설명에 API와 비교 시 주의점을 기록한다. 이후 nominal gain 실험은 reward 함수를 바꾸지 않으며 과거 COM run과 gain이 다르다.

term	이전 RoK4	현재 RoK4	K1
policy period	0.01 s	0.01 s	0.02 s
stand_still_joint_deviation_l1	-0.1	None	없음
feet_standing_contact	없음	-0.2 (최초 -0.1 에서 강화)	별도 항 없음
base_height_l2	-1.0	-5.0	-10.0
hip yaw/roll deviation	-0.05	-0.2	-0.5
hip pitch deviation	-0.005	-0.01	별도 항 없음
action_rate_l2	-0.01	-0.05	-0.10
second_action_rate_l2	-0.005	-0.01	없음

Reward Manager는 각 raw term에 weight와 policy dt 를 곱한다. 그러나 같은 연속 action 궤적에서는 $a_t - a_{(t-1)}$ 자체가 policy period에 비례하므로 action-rate 강도는 weight만으로 직접 비교할 수 없다. RoK4는 K1에 없는 2차 차분 항을 유지하여 $a_t - 2a_{(t-1)} + a_{(t-2)}$ 형태의 고주파 교대 진동을 별도로 억제한다.

```
actor output
-> clip_actions = 1.0
-> clipped_raw_actuator_action
├ last_action observation = clipped_raw_actuator_action
├ action_rate_l2 / second_action_rate_l2
└ scaled_actuator_offset = clipped_raw_actuator_action * ROK4_ACTUATOR_ACTION_SCALE
    └ psi_target = psi_default + scaled_actuator_offset
        └ q_target = J * psi_target
            └ joint_action_target_pos_limits compares q_target with soft_joint_pos_limits
```

여기서 q_{target} 은 PhysX position drive에 직접 입력되는 값이 아니라 explicit actuator model의 입력이다. RoK4AdaptActuator.compute() 가 q_{target} 과 현재 joint state를 actuator 좌표로 변환해 τ_{psi} 를 계산·제한하고, $\tau_q = J^T \tau_{psi}$ 로 바꾼 joint effort만 PhysX에 전달한다. Actuator torque는 최대값의 90%에서 제어 및 reward 기준으로 제한하고, PhysX effort_limit_sim 은 factor를 적용하지 않은 joint mechanical maximum을 최종 solver 안전 상한으로 사용한다.

기존 IsaacGym RoK4 코드는 actuator_actions_raw * action_scale 으로 만든 scaled actuator_actions 를 두 action-rate penalty에 넘겼다. 현재 RoK4Lab은 이 부분만 G1 방식으로 바꾸어 clipped raw action 차분을 사용한다. ROK4_ACTUATOR_ACTION_SCALE 값은 Gym의 좌우 다리 [0.4, 0.5, 1.25, 1.5, 0.75, 0.75] 와 torso 0.4 를 그대로 유지하지만

actuator target 생성에만 사용한다. Observation의 last_action 과 두 action-rate reward가 같은 clipped raw action 좌표를 사용한다.

이 변경은 tensor 크기만 보면 이전 joint-space policy와 같지만 action, joint/actuator position, velocity의 의미가 다르다. 기존 joint-space checkpoint를 resume하면 잘못된 좌표 의미로 학습되므로 actuator-interface checkpoint는 반드시 새 학습에서 생성한다.

Action target의 default_joint_pos 도 Gym의 활성 gait-ready pose와 동일하다. Hip pitch는 -0.0924 rad, knee pitch는 0.345 rad, ankle pitch는 -0.253 rad 이며 나머지 관절은 0.0 rad 이다. 이 joint pose를 $\psi_{\text{default}} = J^{-1} q_{\text{default}}$ 로 변환한 값이 actuator action 중심이며, joint_deviation_* reward는 계속 원래 joint default pose를 기준으로 한다.

Reward와 Domain Randomization의 구분

RoK4RewardsCfg 와 부모 RewardsCfg 는 reward term을 정의한다. 반면 foot friction, restitution, base/upper/lower body mass, base/upper/lower COM, reset pose 같은 domain randomization 값은 domain_randomization_cfg.py 에서 관리한다.

두 설정은 학습 결과에 모두 영향을 주지만 역할은 다르다.

구분	파일	역할
Reward	flat_env_cfg.py 의 RoK4RewardsCfg 및 self.rewards.xxx	policy가 어떤 행동을 더 좋게 볼지 점수 함수를 정의
DR	domain_randomization_cfg.py	물성, 질량, COM, 초기상태 perturbation을 바꿔 다양한 상황에서 버티게 함

예를 들어 발 미끄러짐은 feet_slide reward가 penalty로 줄이고, foot-ground 마찰 계수는 domain_randomization_cfg.py 의 ROK4_STATIC_FRICTION_RANGE 와 ROK4_DYNAMIC_FRICTION_RATIO 가 정한다. 따라서 미끄러짐 문제를 볼 때는 reward 와 DR을 함께 확인해야 하지만, 코드상 관리 위치는 분리되어 있다.

현재 material event는 모든 robot collision shape를 G1/Digit식 nominal $\mu_{\text{static}}=0.8$, $\mu_{\text{dynamic}}=0.6$ 으로 먼저 고정한다. 그 뒤 Foot material DR이 환경마다 $\mu_{\text{static}} \sim \text{Uniform}(0.5, 0.9)$ 를 표본화하고 $\mu_{\text{dynamic}} = 0.75 * \mu_{\text{static}}$ 으로 계산한다. 같은 환경의 좌우 Foot은 동일한 material bucket을 사용하므로 마찰 DR 자체가 좌우 gait 비대칭을 만들지 않으며, dynamic friction은 항상 static friction 이하로 유지된다. Play와 Teleop에서는 이 범위를 사용하지 않고 nominal $0.8/0.6$ 으로 고정해 checkpoint를 재현성 있게 비교한다.

관절 물성은 학습 scene 생성 시 각 environment/joint에 대해 ROK4_STATIC_FRICTION, ROK4_VISCOUS_FRICTION, ROK4_ARMATURE 의 nominal 값을 각각 독립적인 $\text{Uniform}(0.8, 1.2)$ 배율로 scale한다. 이는 PhysX joint property DR이며 actuator-space PD의 ROK4_ACTUATOR_KP/KD gain DR이 아니다. Play와 Teleop에서는 이 joint-physics DR event를 제거해 nominal 관절 물성을 사용한다.

Reward와 PPO entropy/KL의 구분

Reward term은 환경의 RewardManager 가 행동의 점수를 계산하는 항목이다. 반면 PPO의 entropy와 KL은 rsl_rl optimizer가 policy distribution을 업데이트할 때 사용하는 학습 지표다. 따라서 entropy_coef 나 desired_kl 은 RoK4RewardsCfg 의 reward weight가 아니다.

Reward와 PPO 지표 비교

항목	현재 값/위치	역할
reward weight	flat_env_cfg.py 의 RewTerm(weight=...)	환경 행동의 positive reward 또는 penalty 크기를 결정
entropy_coef	rsl_rl_ppo_cfg.py 의 0.002	Gaussian policy의 exploration entropy를 유지하려는 optimizer 압력을 결정
desired_kl	rsl_rl_ppo_cfg.py 의 0.01	old/new policy 차이를 기준으로 adaptive learning rate를 조절

RoK4 로컬 scripts/rsl_rl/rok4_ppo.py 는 adaptive schedule에서 이미 계산되는 mini-batch KL을 모아 TensorBoard의 Loss/kl``(iteration 평균)과 ``Loss/kl_max``(iteration 최대)로 기록한다. 현재 5 learning epochs와 4 mini-batches를 사용하므로 iteration당 20개 KL 값을 집계한다. 이 로깅을 위해 Isaac Lab 또는 설치된 ``rsl_rl 파일을 수정하지 않는다.

Reward 계산 방식

Isaac Lab의 RewardManager 는 각 reward term의 raw value를 계산한 뒤 weight 를 곱하고, 모든 term을 합산한다.

```
total_reward =
    sum(weight_i * reward_function_i(env, params_i))
```

따라서 positive weight는 행동을 유도하고, negative weight는 penalty로 작동한다.

RoK4 전용 Reward Terms

아래 term들은 RoK4RewardsCfg 에서 직접 정의하거나 부모 RewardsCfg 의 term을 RoK4용으로 override한 항목이다.

term	function	weight	params	의미
termination_penalty	mdp.is_terminated	-200.0	없음	episode가 termination되면 큰 penalty를 준다.
track_lin_vel_xy_exp	mdp.track_lin_vel_xy_yaw_frame_exp	1.0	command_name="base_velocity", std=0.5	yaw-aligned robot frame에서 x/y 선속도 command를 추종하게 한다.
track_ang_vel_z_exp	mdp.track_ang_vel_z_world_exp	1.0	command_name="base_velocity", std=0.5	world frame yaw angular velocity command를 추종하게 한다.
base_height_l2	mdp.base_height_relative_l2	-5.0	flat environment origin 기준 target_height=0.907 m	root world Z에서 각 environment origin Z를 뺀 base height가 gait-ready IK 기준 높이와 달라지는 정도를 제곱 penalty로 만든다. K1의 -10.0 을 그대로 복제하지 않고 절반 강도로 자체 유지 효과와 보행 경직을 함께 확인한다.
feet_air_time	mdp.FeetAirTimeTouchdownBiped	2.0	feet: L_Foot_Link, R_Foot_Link; target_air_time=0.50 s, command_threshold=0.05 m/s	정확히 한 발이 first contact가 된 step에 완료된 air-time의 T - 0.50 을 한 번 지급하고 같은 유효 touchdown의 실제 T 를 metric으로 누적한다. 최대 보상 cap은 없으며 stateless 함수 버전도 비교용으로 남아 있다.
feet_clearance	mdp.feet_swing_clearance_exp	0.2	Foot body-origin target 0.054 m, std 0.04 m, velocity scale 0.50 m/s, yaw lift fraction 0.50. 현재 train collision sole은 local Z=0이므로 별도 4 mm를 빼지 않는다.	선형 명령에서는 yaw frame의 command 방향 진행속도만 tanh gate에 사용한다. Pure-yaw에서는 높이 점수 50%와 명령 회전 점선 방향 진행 점수 50%를 합쳐 수직 lift를 허용하되 임의의 전후 swing은 추가 보상하지 않는다. Air-time 길이는 touchdown reward가 별도로 담당한다.
no_jumps	mdp.desired_contacts	-2.0	feet: L_Foot_Link, R_Foot_Link; force threshold=1.0 N	최근 contact-force history에서 양발 모두 threshold를 넘는 접촉이 없을 때만 raw penalty 1 을 반환한다. 정상 single stance와 toe-off는 허용하지만 양발 동시 flight phase를 억제한다.

term	function	weight	params	의미
feet_slide	mdp.feet_slide	-0.2	feet: L_Foot_Link, R_Foot_Link	지면 접촉 중인 발이 미끄러지는 것을 줄인다.
feet_touchdown_velocity	mdp.FeetTouchdownVelocityL2	-10.0	safe landing velocity 0.0 m/s	이전 policy step의 공중 발 world-XYZ 속도를 저장한다. First contact가 발생하면 $v_x_{prev}^2 + v_y_{prev}^2 + \text{relu}(-v_z_{prev})^2$ 을 발별로 합산해 사건당 한 번 penalty로 만든다.
feet_contact_velocity	mdp.feet_contact_velocity_l2	None (현재 비활성)	실험값: landing height 0.03 m, approach threshold -0.6 m/s, impact threshold 0.0 m/s, weight -10.0	기존 Gym의 접근/착지 velocity penalty 함수는 비교용으로 남아 있지만, 접근 구간까지 shaping해 보행 형태가 변한 실험 이후 활성 term에서 제외됐다.
feet_touchdown_edge_velocity	mdp.FeetTouchdownEdgeVelocityL2	-0.1	고정 landing window 0.10 s, pre_touchdown_scale=5.0; collision sole 네 모서리	발마다 가장 빠른 모서리 하강 Z 속도를 제공한다. 직전 sample 비용만 5배, 접촉부터 현재 sample 10회는 기존대로다. COM 항은 유지한다. 수평속도/각도 목표/command gate는 없다.
feet_touchdown_pitch_l2	mdp.FeetTouchdownPitchL2	-1.0	이전/현재 policy sample의 yaw-removed 발 법선 X 성분 제공	직전 sample이 공중인 발의 첫 접촉에서 두 오차 중 큰 값을 발별 합산한다. 기존 swing pitch는 -0.1로 유지한다. 지속 지지/일반 swing/toe-off에는 새 비용이 없으며 reset 초기 접촉은 제외한다.
feet_contact_force	mdp.FeetContactForceL2	None (현재 비활성)	후속 비교 실험값: force limit 1.2 x randomized body weight, landing window 0.10 s, weight -0.05	함수는 비교용으로 남아 있으나 현재 학습 reward에는 연결하지 않는다.
feet_contact_force_metrics	mdp.FeetContactForceL2 (metric_only=True)	1.0 (함수 반환값은 항상 0)	landing window 0.10 s	Reward Manager가 매 step 호출하여 touchdown peak GRF를 집계하지만 reward 합에는 정확히 0을 더한다. TensorBoard Metrics/feet_touchdown/mean_peak_normal_force 만 기록한다.
feet_touchdown_diagnostics	mdp.FeetTouchdownDiagnostics	1.0 (함수 반환값은 항상 0)	sole corners: toe x=0.175 m, heel x=-0.060 m, half width 0.045 m; force windows 0-20 ms, 20-100 ms	발 원점 선행속도에 $\omega \times r$ 회전속도를 더해 toe/heel/최저 모서리의 착지 직전 속도를 기록한다. World-Z normal-force peak도 초기 충돌과 이후 체중 인수 구간으로 나누며 reward에는 영향을 주지 않는다.
feet_touchdown_acc	mdp.feet_touchdown_acc	None (현재 비활성)	실험값: acceleration threshold 50 m/s ² , weight -0.002	ROBOTIS KI과 같은 함수는 남아 있지만 touchdownc50 실험에서 MuJoCo와 실기 착지 충격이 개선되지 않아 비활성화했다. First-contact의 10 ms 판정 구간과 마지막 2 ms physics sample의 가속도 시점이 일치하지 않을 수 있다.
feet_flat_orientation_l2	mdp.feet_flat_orientation_l2	None (현재 비활성)	좌우 Foot body quaternion과 contact sensor	함수는 접촉 중인 발의 local +Z 축을 world frame으로 회전한 뒤 X/Y 성분 제공합을 계산한다. 현재는 world-up 수평 제약이 toe-off와 향후 경사면 적응을 방해하지 않도록 Reward Manager에 등록하지 않는다.
feet_swing_roll_l2	mdp.feet_swing_roll_l2	-1.0	좌우 Foot body quaternion과 contact sensor	공중에 있는 발의 local +Z 법선을 각 Foot 자체의 yaw frame으로 변환하고 lateral 성분의 제공만 계산한다. 따라서 swing-foot의 안쪽/바깥쪽 roll을 억제하고 지지 발 자세는 제한하지 않는다.
feet_swing_pitch_l2	mdp.feet_swing_pitch_l2	-0.1	좌우 Foot body quaternion과 contact sensor	같은 yaw-removed sole normal의 forward 성분 제공을 swing 발에만 적용한다. Roll 항의 1/10 가중치로 toe-up/toe-down 기울임을 억제한다. 지지 발에는 적용하지 않으며 heel-first 착지를 강제하지 않는다.
feet_stance_width_l2	mdp.feet_stance_width_l2	None (현재 비활성)	목표 0.21 m, hard minimum 0.18 m; 직진 이동 command에서만 활성화	함수는 좌우 Foot 위치 차이를 base yaw frame으로 회전해 lateral width를 계산한다. 0.21 m 초과 폭은 coefficient 0.5로 억제하고, 0.18 m 미만 또는 발 교차는 coefficient 5.0으로 강하게 억제한다. 현재 RoK4RewardsCfg에서는 term을 None으로 두어 Reward Manager에 등록하지 않는다.
feet_lateral_separation_l2	mdp.feet_lateral_separation_l2	-2.0	experimental minimum width 0.165 m; feet 순서는 Left, Right	좌우 Foot 위치 차이를 base yaw frame으로 회전한 뒤 signed lateral width $y_{left} - y_{right}$ 를 유지한다. $\text{relu}(0.165 - \text{signed_width})^2$ 만 반환하므로 정상 0.21 m 폭과 넓은 폭은 제한하지 않고, 좁아질수록 penalty가 증가하며 좌우 Foot이 교차해 부호가 바뀌면 더 크게 작동한다. Command mask 없이 standing과 moving 모두에 적용된다. 이 값은 실기 발목 부품 간 여유를 확인하기 위해 기존 0.16 m에서 5 mm 증가시킨 soft penalty 하한이며, 기하학적 hard constraint는 아니다.
stand_still_joint_deviation_l1	mdp.stand_still_joint_deviation_l1	None (현재 ablation; rollback -0.1)	ROK4_JOINT_ORDER 전체 13관절, base_velocity.is_standing_env mask	함수는 RoK4 command generator가 standing으로 지정한 환경에서 실제 joint position과 default joint position 차이의 절댓값 합을 반환한다. 현재는 term을 등록하지 않고 별도의 양발 접촉 항을 시험한다. 검증된 -0.1을 복원 기준으로 보존한다. 기존 Gym의 defaultPosStanding에 대응한다.
feet_standing_contact	mdp.feet_standing_contact	-0.2	command_name="base_velocity"; 좌우 Foot contact sensor	세 command가 정확히 0일 때만 미접촉 발 수를 반환한다. 양발/한발/무접촉은 raw 0/1/2이며 자세, 발 위치, 하중 분배는 요구하지 않는다. 정지 중 보호 스텝에도 적용되므로 회복 동작 저하를 확인한다.
dof_pos_limits	mdp.joint_pos_limits	-1.0	ROK4_JOINT_ORDER에 포함된 전체 13관절, preserve_order=True	각 관절이 USD hard limit의 95%로 계산된 soft joint limit을 넘는 양을 합산해 억제한다.
joint_action_target_pos_limits	mdp.joint_action_target_pos_limits	-0.001	actuator_pos action의 mapped joint target과 ROK4_JOINT_ORDER의 전체 13관절	Gym의 penalty_action_limits에 대응한다. ADAPT mapping 뒤 목표 관절 위치가 95% soft limit을 넘는 양을 합산한다.
joint_deviation_hip	mdp.joint_deviation_l1	-0.2	.*_Hip_Yaw_Joint, .*_Hip_Roll_Joint	RoK4의 회전된 hip joint frame에서는 lateral foot placement가 이름상 yaw/roll 두 축의 조합으로 생성된다. 첫 no-standing-pose 실험의 -0.1보다 두 배 강화해 과도한 편차를 줄여본다. 이 항은 standing/moving 모두에 적용되는 기본 자세 편차 비용이며 발 heading 각도 자체의 penalty가 아니다. 회복과 recovery step을 과도하게 제한하는지도 확인한다.

term	function	weight	params	의미
joint_deviation_hip_pitch	mdp.joint_deviation_l1	-0.01	. *_Hip_Pitch_Joint	전후진 및 recovery 보폭을 만드는 핵심 관절이므로 yaw/roll의 1/20 가 중치 크기를 유지하면서, default pose에서 과도하게 벗어나 다리 전체를 크게 휘두르는 전략을 약하게 억제한다. Swing phase나 무릎 굽힘을 직접 판정하는 reward는 아니다.
joint_deviation_torso	mdp.joint_deviation_l1	-0.1	Torso_Yaw_Joint	torso yaw가 default pose에서 과도하게 벗어나지 않게 한다.
actuator_acc_l2	mdp.actuator_acc_l2	-1.0e-8	전체 RoK4 actuator; index 2,3,8,9 weight 0.5	Isaac Lab의 joint acceleration을 J^{-1} 로 actuator acceleration으로 변환한다. 실제 rad/s^2 값이며 Gym의 undivided velocity difference와는 다르다.
actuator_torques_l2	mdp.actuator_torques_l2	-2.0e-6	전체 RoK4 actuator; index 2,3,8,9 weight 0.5	ADAPT explicit actuator가 torque-limit clip 뒤 보관한 τ_{psi} 의 weighted squared sum이다.
actuator_vel_l2	mdp.actuator_vel_l2	-1.0e-4	전체 RoK4 actuator; index 2,3,8,9 weight 0.5	$\dot{\text{psi}} = J^{-1} \dot{q}$ 의 weighted squared sum으로 Gym penalty_joint_vel 에 대응한다.
actuator_velocity_limits	mdp.actuator_velocity_limits	-0.001	velocity_limit_factor=0.9 를 적용한 actuator velocity limits	$\text{abs}(\dot{\text{psi}})$ 가 설정된 velocity limit을 넘은 양을 합산한다.
actuator_torque_limits	mdp.actuator_torque_limits	-1.0e-5	torque_limit_factor=0.9 를 적용한 actuator torque limits	clip 전 요청 τ_{psi} 가 설정된 torque limit을 넘은 양을 합산해 saturation 요구를 표시한다.
action_rate_l2	mdp.action_rate_l2	-0.05	clipped raw action 전체 13차원; index 2,3,8,9 weight 0.5	$a_t - a_{t-1}$ 의 weighted 제곱합을 줄인다. 이전 -0.01 보다 다섯 배 강화했지만 100 Hz RoK4의 step 간 action 차이는 50 Hz K1보다 작으므로 K1 -0.10 의 단순 절반과 물리적으로 동일하지 않다. Action scale은 적용하지 않는다.
second_action_rate_l2	mdp.second_action_rate_l2	-0.01	clipped raw action 전체 13차원; index 2,3,8,9 weight 0.5	raw action의 2차 차분, 즉 $a_t - 2a_{t-1} + a_{t-2}$ 의 weighted 제곱합으로 action curvature와 고주파 교대 진동을 완화한다. K1에는 없는 RoK4 전용 항으로 유지한다. reset 직후 첫 두 policy step 은 history가 부족하므로 0으로 처리한다.

Gym과 Lab의 action-limit 신호 차이

Gym과 현재 Lab 구현 모두 actuator action을 ADAPT 변환한 mapped joint target을 검사한다. 현재 Lab의 RoK4ActuatorPositionAction.processed_actions 가 $q_{\text{target}} = J \psi_{\text{target}}$ 을 보관하므로 joint_action_target_pos_limits 는 실제 PD command에 대응하는 joint target을 검사한다.

Gym과 Lab의 torque/velocity 신호 차이

기존 Gym 코드와 현재 Lab 코드는 모두 ADAPT transmission 변환 전 actuator torque/velocity를 penalty에 사용한다. 현재 Lab custom actuator는 applied_actuator_effort 를 직접 노출하고 joint velocity를 J^{-1} 로 변환한다. 따라서 torque/velocity reward 좌표계와 0.5 weighting은 Gym과 동일하다. Acceleration만 현재 Lab에서 $J^{-1} \ddot{q}$ 의 실제 acceleration을 사용하므로 Gym의 $\dot{\psi}_t - \dot{\psi}_{t-1}$ 과 정의가 다르다.

부모 RewardsCfg에서 상속받고 RoK4에서 수정한 Terms

아래 term들은 RewardsCfg 에서 기본으로 제공되며, RoK4FlatEnvCfg.__post_init__() 에서 weight나 body/joint 대상이 RoK4에 맞게 수정된다.

term	function	current weight	RoK4 설정	의미
lin_vel_z_l2	mdp.lin_vel_z_l2	-0.2	기본 robot root	base가 z 방향으로 튀는 움직임을 줄인다.
ang_vel_xy_l2	mdp.ang_vel_xy_l2	-0.05	기본 robot root	roll/pitch angular velocity를 줄인다.
flat_orientation_l2	mdp.flat_orientation_l2	-5.0	기본 robot root	projected gravity의 X/Y 성분 제곱합으로 물이 기울어지는 것을 줄인다. -1.0 과 -2.0 실험 뒤 -5.0 으로 크게 강화하여 lateral hopping이 과도한 body roll 전략인지 확인하고, 자연스러운 체중 이동과 속도 추종을 과도하게 제한하는지도 함께 관찰한다.

term	function	current weight	RoK4 설정	의미
undesired_contacts	mdp.undesired_contacts	-1.0	Base_Link, Upper_Body_Link; threshold=1.0	발이 아닌 base/upper body 접촉을 penalty로 처리한다.

Reward Function 요약

function	위치	계산 의미
is_terminated	Isaac Lab 공통	termination manager의 terminated flag를 reward term으로 반환한다.
track_lin_vel_xy_yaw_frame_exp	locomotion mdp	yaw frame x/y 선속도 command error에 $\exp(-\text{error} / \text{std}^2)$ 를 적용한다.
track_ang_vel_z_world_exp	locomotion mdp	world z축 angular velocity command error에 $\exp(-\text{error} / \text{std}^2)$ 를 적용한다.
base_height_relative_l2	RoK4 local mdp	flat environment origin 기준 root 높이와 0.907 m 목표의 제곱 오차를 반환한다.
feet_air_time_touchdown_biped	RoK4 local mdp	정확히 한 발이 first contact가 된 순간 $\text{last_air_time} - \text{target_air_time}$ 을 한 번 반환하는 상한 없는 stateless 함수다.
feet_swing_clearance_exp	RoK4 local mdp	선형 명령에서는 유효 swing Foot별 $\tanh(\text{relu}(\text{v_foot} \cdot \text{yaw_unit}(\text{command_xy}))/0.50) \cdot \exp(-(h-0.054)^2/0.04^2)$ 를 평균한다. Pure-yaw에서는 높이 gate 0.5 와 명령 회전 점선 방향 진행 gate 0.5 를 합치며, standing과 zero command에서는 0이다.
desired_contacts	Isaac Lab 공통	지정 body 중 하나라도 최근 force history에서 threshold를 넘으면 0, 모두 접촉하지 않으면 1을 반환한다. Negative weight와 함께 양발 flight penalty로 사용한다.
feet_slide	locomotion mdp	접촉 중인 foot의 horizontal linear velocity norm을 penalty로 계산한다.
feet_contact_velocity_l2	RoK4 local mdp	비활성 비교용 함수다. 접촉 전에는 $I(h<0.03) \cdot I(v_z<-0.6) \cdot v_z^2$, first contact에서는 $I(v_z<0) \cdot v_z^2$ 를 발별로 합산한다.
feet_contact_force_l2	RoK4 local mdp	비활성 비교용 함수다. 5-sample 범선력 history의 peak에 대해 $\text{sum}(w_contact \cdot (\text{relu}(F_peak-1000)/1000)^2)$ 를 반환한다. $w_contact$ 는 stance 0.15, first contact 1.0 이다.
FeetTouchdownVelocityL2	RoK4 local mdp	이전 policy sample의 공중 발 하강속도 $s=\text{relu}(-v_z, \text{previous})$ 를 저장하고, first contact에서만 $\text{sum}(s^2)$ 를 반환한다. Reward Manager reset 시 history를 무효화해 초기 접촉을 제외한다.
feet_touchdown_acc	RoK4 local mdp	first-contact인 발에 대해서만 $\text{relu}(a_foot, w - \text{threshold})$ 를 합산하는 ROBOTIS K1 호환 함수다. 현재 Reward Manager term은 None 이며 함수와 단위 테스트만 비교용으로 유지한다.
lin_vel_z_l2	Isaac Lab 공통	base body-frame z velocity squared.
ang_vel_xy_l2	Isaac Lab 공통	base body-frame roll/pitch angular velocity squared sum.
flat_orientation_l2	Isaac Lab 공통	projected gravity의 x/y component squared sum으로 몸 기울기를 penalty화한다.
actuator_torques_l2	RoK4 로컬 mdp	actuator-space applied torque weighted squared sum. Index 2,3,8,9 는 weight 0.5.
actuator_vel_l2	RoK4 로컬 mdp	$J^{-1} \cdot \dot{q}$ actuator velocity의 weighted squared sum. Index 2,3,8,9 는 weight 0.5.
actuator_acc_l2	RoK4 로컬 mdp	$J^{-1} \cdot \ddot{q}$ actuator acceleration의 weighted squared sum. Index 2,3,8,9 는 weight 0.5.
actuator_velocity_limits	RoK4 로컬 mdp	velocity_limit_factor 를 적용한 actuator velocity limit 초과량.
actuator_torque_limits	RoK4 로컬 mdp	clip 전 actuator torque command가 torque_limit_factor 적용 limit를 넘은 양.
action_rate_l2	RoK4 로컬 mdp	현재 raw action과 이전 action 차이의 weighted squared sum. 함수 내부 clamp는 없지만 표준 RoK4 runner가 입력을 먼저 [-1,1] 로 제한한다. Index 2,3,8,9 는 weight 0.5.
second_action_rate_l2	RoK4 로컬 mdp	raw action의 2차 차분 weighted squared sum. 표준 RoK4 runner 경로에서는 wrapper-clipped raw action이다. Index 2,3,8,9 는 weight 0.5.
joint_deviation_l1	Isaac Lab 공통	현재 joint position과 default joint position 사이의 absolute sum.
stand_still_joint_deviation_l1	RoK4 로컬 mdp	base_velocity.is_standing_env 가 true일 때 전체 13관절의 default-pose absolute error sum을 계산한다. 현재 ablation에서는 term이 None 이며 rollback 기준은 -0.1 이다.
joint_pos_limits	Isaac Lab 공통	soft joint position limit을 넘은 정도를 합산한다.
joint_action_target_pos_limits	RoK4 로컬 mdp	actuator_pos action이 ADAPT mapping한 joint target의 soft joint position limit 초과량.
undesired_contacts	Isaac Lab 공통	지정 body의 contact force가 threshold를 넘은 contact 개수를 penalty로 반환한다.

Command와 Reward의 연결

현재 RoK4 flat command range는 다음과 같다.

```
lin_vel_x = (-0.3, 0.85)
lin_vel_y = (-0.3, 0.3)
ang_vel_z = (-0.6, 0.6)
```

RoK4PeriodicFreezeVelocityCommand 는 episode reset마다 환경 역할을 다음 확률로 표본화한다.

1. 35% mixed: vx, vy, wz 를 모두 표본화한다.
2. 5% standing: episode 동안 exact-zero command를 유지한다.
3. 5% walking: planar command norm 0.10 m/s 이상만 사용하고 freeze하지 않는다.
4. 20% x: vx= $\pm$ Uniform(0.10, 0.30) m/s 저속 대칭 전후진만 표본화한다.
5. 5% fast_forward: vx=Uniform(0.30, 0.85) m/s 고속 전진만 표본화한다.
6. 10% y: lateral 좌/우 command만 표본화한다.
7. 10% yaw: 시계/반시계 yaw-rate command만 표본화한다.
8. 10% x_yaw: vy=0 인 저속 대칭 전후진+회전 command를 표본화한다.

코드에서는 command 축과 직접 대응하는 x/y/yaw/x_yaw 이름을 사용한다. 각 단일 축 역할은 양/음 부호를 50:50 확률로 표본화하며 x_yaw 의 두 부호는 독립적으로 표본화한다. x 와 x_yaw 의 vx 는 절댓값 0.10~0.30 m/s 로 대칭이고, fast_forward 가 0.30~0.85 m/s 를 별도로 담당한다. vy 의 최소 절댓값은 0.10 m/s, wz 는 0.10 rad/s 이다. 4096개 환경에서 20% x 역할은 평균 약 819개이고, 각 부호는 평균 약 410개다. 나머지 10% 전용 역할은 각각 평균 약 410개다.

mixed/x/fast_forward/y/yaw/x_yaw 의 비율 합 90%는 10.0 s cycle과 환경별 Uniform(1.5, 3.0) s freeze duration을 사용한다. Random initial phase와 독립 duration 때문에 같은 PPO batch에서 moving, standing, transition을 동시에 관측한다. Episode 중 command를 다시 표본화할 때는 역할을 유지하며, reset 때 역할 자체를 다시 무작위로 뽑는다. 부모의 별도 standing 확률은 rel_standing_envs=0.0 으로 비활성화한다. 작은 non-zero mixed command는 standing으로 바꾸지 않고 연속적인 저속 이동 목표로 유지한다.

여기서 random initial phase는 global simulation 시작 시 한 번만 주는 값이 아니다. 각 environment가 reset될 때마다 freeze phase를 Uniform(0, 10) s 로 다시 표본화하므로 reset 직후 freeze로 시작할 수도 있다. 학습 episode timeout은 환경별 20 s 이며, early termination이 난 환경만 episode counter, episode role, freeze phase를 다시 시작한다. 예를 들어 global time 3 s 에 reset된 환경은 이후 20초를 생존하면 약 23 s 에 timeout된다. 초기에는 episode counter가 같지만 early reset이 누적되면서 환경별 episode phase가 자연스럽게 달라진다.

RoK4-local training RoK4MixedPush 는 freeze와 다른 환경별 timer다. Reset마다 다음 event를 Uniform(10, 15) s 에서 표본화한다. Base-yaw Delta vx 는 -0.5~1.0 m/s, Delta vy 는 -0.5~0.5 m/s 이며, event의 50%는 root velocity에 즉시 더하고 50%는 0.05~0.50 s 동안 $F = m \cdot \Delta v / T$ force pulse로 적용한다. 따라서 외란은 이동 command 중에도, exact-zero freeze 중에도, 전환 부근에도 발생할 수 있다. Freeze 중 외란이 들어와도 command와 is_standing_env 는 zero/true로 유지된다. 현재 ablation에서는 standing 전용 joint-deviation term 없이 dense velocity-tracking, 일반 동작 비용과 새 feet_standing_contact 로 복구와 정지를 평가하며, 기존 pose -0.1 을 rollback 기준으로 유지한다. Contact reward와 no_jumps 는 같은 구간의 실제 발 접촉 상태를 그대로 평가한다. Play/Teleop에서는 자동 event를 끄고 수동 velocity Push Test UI를 사용한다.

Periodic freeze와 standing 역할은 command를 [0, 0, 0] 으로 만드는 동시에 is_standing_env=True 를 설정한다. 이 mask는 feet-air-time과 clearance 같은 standing-aware gait term 및 stand_still_joint_deviation_l1 에 사용할 수 있다.

현재 ablation은 이 term을 None 으로 두고, 새 접촉 항에서 실제 command의 exact-zero 여부를 직접 검사한다. 검증된 pose -0.1 설정은 exact-zero에서 default pose로 향하는 편향이 다시 필요할 때 복원한다.

정지 양발 접촉 패널티 (2026-09-08)

비교 checkpoint는 2026-09-07_15-53-13_concurrent_estimator_klstyle_nostand_bh5_hip020_hippitch001_ar05_ar2_01_fresh25k/model_24999.pt 이다. 이번 변경은 기본 자세로 복귀시키지 않고 정지 중 반복적인 single support를 직접 억제해보는 실험이다. feet_air_time 과는 별도 함수/term이며 base-height -5.0, base-Z velocity -0.2, hip deviation, action-rate, gain, estimator, command/freeze/push 구성은 그대로다. feet_standing_contact=None 으로만 바꾸면 이 새 목적함수를 비활성화할 수 있다.

최초 -0.1 run은 2026-09-08_13-40-45_concurrent_estimator_hip020_standcontact010_fresh25k 이다. 사용자는 model_5000.pt Teleop에서 제자리 stepping을 확인했고, 직전 500회 평균 접촉 cost는 5000에서 -0.02365, 7122에서 -0.02318 이었다. 다음 실험은 가중치만 -0.1 -> -0.2 로 바꿨고, 사용자는 그 run의 마지막 checkpoint에서 제자리 stepping 해소와 보행 유지를 확인했다. 위 비교 기록은 Isaac Sim 관찰이며 실기 검증을 뜻하지 않는다. 링크 원점 실험과 COM 복원 모두 아래 수식과 표의 -0.2 를 유지한다.

```
c_i = 1[current_contact_time_i > 0] (i = L, R)
I_stop = 1[(vx == 0) and (vy == 0) and (wz == 0)]
P_support = I_stop * ((1 - c_L) + (1 - c_R))
r_support = -0.2 * P_support * dt
```

vx, vy 는 m/s, wz 는 rad/s 이다. 서로 단위가 다른 세 값을 하나의 norm threshold로 묶지 않고 각 성분이 정확히 0인지 본다. 따라서 작은 non-zero 이동, pure-yaw, 선행+회전 명령에는 이 항이 작동하지 않는다. 새 command deadzone이나 stable/recovery threshold는 추가하지 않는다.

정지 명령에서의 기여

현재 접촉 상태	Raw penalty	Step reward (dt = 0.01 s)
양발 접촉	0	0
한발 접촉	1	-0.002
양발 미접촉	2	-0.004

접촉 판정은 기존 ContactSensor 의 track_air_time=True 와 현재 접촉 시간 buffer를 사용한다. 새 GRF 크기 threshold나 최근 force history 최대값을 계산하는 항이 아니다. Stateless 함수이므로 별도 class, reset state, per-step CPU logging 또는 history buffer가 없다. 양발 센서 선택이 아니거나 air-time tracking이 꺼져 있으면 명시적으로 오류를 낸다.

이 항은 touchdown 순간만이 아니라 정지 명령 동안 매 step 적용된다. Recovery로 발 위치를 바꿔도 다시 양발이 접촉하면 이 항의 벌점은 0이다. 그러나 보호 스텝을 드는 동안에는 비용을 받으므로 외란 복구나 급정거를 방해할 수 있다. 또한 발끝만 닿아도 접촉으로 인정될 수 있고, 양발 하중 균등/발바닥 수평/미끄럼 방지를 요구하지 않는다. no_jumps 는 유지하므로

정지 중 flight는 두 항에서 동시에 벌점을 받을 수 있다. Quiet standing뿐 아니라 횡보/회전/push recovery, toe dragging을 같은 command 조건에서 함께 검증한다.

TensorBoard의 Episode_Reward/feet_standing_contact 는 Reward Manager가 자동 기록한다. 새 custom metric은 없다. 이 값은 weighted episode sum을 최대 episode 길이 20 s 로 나눈 값이며, 정지 구간에 조건부로 계산한 양발 지지율은 아니다. 20초 전체가 정지 명령이며 한발 지지만 유지하면 약 -0.2, flight만 유지하면 약 -0.4, 양발 접촉만 유지하면 0 이다. Moving 시간이 많거나 조기 종료되어도 절댓값이 작아질 수 있으므로, 로그가 0에 가까워졌다는 사실만으로 제자리 stepping 해결을 주장하지 않는다.

가중치를 바꾼 두 run을 비교할 때는 각각의 Episode_Reward/feet_standing_contact 를 그 run의 signed weight로 나눈다. 동일한 미접촉 노출이면 weight=-0.1 의 로그 -0.02 와 weight=-0.2 의 로그 -0.04 는 모두 weight-normalized cost 0.2 다. 이 보정 후에도 정지 시간/조기 종료 차이는 남으므로 quiet standing과 보호 스텝 유지 여부를 별도로 확인한다.

기존 command-aware 보상 (변경 없음)

이 command는 track_lin_vel_xy_exp, track_ang_vel_z_exp, feet_air_time 에 직접 영향을 준다. 특히 FeetAirTimeTouchdownBiped 는 x/y command norm이 0.05 이하이면 reward를 0으로 만든다. 즉 거의 정지 명령에서는 stepping reward가 강하게 작동하지 않는다.

UniformVelocityCommand 가 반환하는 [lin_vel_x, lin_vel_y, ang_vel_z] 는 robot base frame 기준 command다. 따라서 양의 lin_vel_x 는 로봇이 현재 바라보는 방향의 전진, 양의 lin_vel_y 는 로봇 기준 좌측 이동, 양의 ang_vel_z 는 z축 주위 반시계 방향 회전을 뜻한다. 선속도 tracking 함수는 실제 world 선속도에서 roll/pitch를 제외한 yaw 회전만 역변환한 gravity-aligned yaw frame 속도와 x/y command를 비교한다. 반면 현재 각속도 tracking 함수는 command의 z 성분과 world-frame root_ang_vel_w[:, 2] 를 비교한다. 평평하고 직립한 상태에서는 두 z축이 거의 같지만, 몸체가 크게 기울면 완전히 동일한 표현은 아니다.

RoK4는 부모 G1-style heading mode를 사용하지 않는다. heading_command=False, rel_heading_envs=0.0, heading=None 으로 설정하고 ang_vel_z=(-0.6, 0.6) rad/s 에서 목표 wz 를 직접 표본화한다. 따라서 학습, Gym, Teleop, ROS cmd_vel 과 실물 적용에서 policy가 받는 세 번째 command는 모두 같은 의미의 목표 yaw angular velocity다. Navigation에서 목표 world heading이 필요하면 policy 외부의 상위 controller가 heading error를 wz 로 변환해 이 동일한 velocity interface에 전달한다.

현재 feet_air_time 은 느리고 긴 step을 유도하기 위해 target_air_time=0.50 s, weight=2.0 을 사용한다. 정확히 한 발이 first contact가 된 policy step에 그 발의 완료된 last_air_time 을 읽고 last_air_time - 0.50 을 한 번 반환한다. 양발이 같은 step에 first contact가 되거나 planar command norm이 0.05 m/s 이하이면 0이다. 따라서 T=0.30 s 는 raw -0.20, T=0.50 s 는 0, T=0.75 s 는 raw +0.25 이다. Reward Manager가 적용하는 pre-dt weighted event는 각각 raw 값에 2.0 을 곱한다. Policy interval 0.01 s 까지 포함한 touchdown-error 기울기는 0.02 로, K1의 1.0 * 0.02 와 같다.

이 signed touchdown shaping은 짧은 잔걸음을 단순히 적게 보상하는 대신 직접 penalty화한다. 반면 최대 보상 air-time cap 이 없으므로 0.75 s 를 초과한 완료 swing도 더 큰 양수를 받는다. 한 발을 오래 드는 전략은 velocity tracking, no_jumps 와 기타 gait term이 함께 제한해야 한다. 이 sparse event term의 TensorBoard 값은 이전 dense 또는 squared touchdown term과 직접 비교할 수 없다.

현재 활성 term은 stateful FeetAirTimeTouchdownBiped class다. Reward Manager가 기록하는 Episode_Reward/feet_air_time 은 episode 동안 합산된 weighted reward이므로 실제 평균 air time 자체와 같지 않다. Class는 touchdown event의 last _air_time 합과 event count를 GPU tensor로 누적하고 reset 때 Metrics/feet_touchdown/mean_air_time [s]를 기록한다. base_height_l2 는 평지에서 height scanner 없이 각 environment origin을 지면 기준으로 사용한다.

$$p_{base - height} = (z_{base, w} - z_{origin, w} - 0.907)^2$$

Raw 값에는 weight=-5.0 과 policy dt 가 적용된다. 목표는 standing에서만이 아니라 전체 보행에 적용되므로 몸통의 자연스러운 상하 진동도 penalty를 받는다. 이는 이전 -1.0 보다 다섯 배 강하고 K1 -10.0 의 절반인 중간 실험값이다. 평균 자세 유지가 개선되는지와 보행 경직 및 착지 충격이 증가하는지를 함께 확인한다.

feet_clearance 는 touchdown event가 아니라 swing 중 매 policy step 계산하는 dense positive reward다. 유효한 Foot 집합을 S 라고 하고 높이 score를 H_i 라고 하면 다음과 같다.

$$H_i = \exp\left(-\frac{(h_i - 0.054)^2}{0.04^2}\right)$$

선형 command c_xy 가 있으면 기존과 동일하게 command 진행 방향 성분만 사용한다.

$$G_{lin, i} = \tanh\left(\frac{\max(0, v_{i, xy}^{yaw} \cdot \hat{c}_{xy})}{0.50}\right), r_{clear} = \frac{1}{|S|} \sum_{i \in S} H_i G_{lin, i}$$

Pure-yaw command에서는 root에서 Foot으로 향하는 yaw-frame 반경 r_i 로 각 Foot의 명령 회전 접선 방향을 계산한다. Base의 평행이동을 회전 진행으로 오인하지 않도록 Foot 속도에서 root 선속도를 뺀다.

$$\hat{d}_{yaw, i} = \text{sign}(\omega_z) \frac{[-r_{i, y}, r_{i, x}]}{\|[-r_{i, y}, r_{i, x}]\|},$$

$$G_{yaw,i} = 0.5 + 0.5 \tanh\left(\frac{\max(0, (v_i - v_{root})_{xy}^{yaw} \cdot \hat{d}_{yaw,i})}{0.50}\right), \quad r_{clear} = \frac{1}{|S|} \sum_{i \in S} H_i G_{yaw,i}$$

따라서 pure-yaw에서 목표 높이만 맞춘 수직 lift는 $0.5 H_i$ 를 받고, 올바른 접선 진행이 커질수록 H_i 에 접근한다. 반대 접선 방향 이동은 progress bonus 없이 $0.5 H_i$ 에 머문다. 선형 command와 yaw command가 모두 없으면 gate와 reward는 0 이다. $h_i = z_{FootLink,i,w} - z_{origin,w}$ 이다. S 에는 $current_air_time > 0$ 이고 command generator의 $is_standing_env=False$ 인 발만 포함된다. 유효한 발이 없으면 0이다. 두 발이 동시에 유효해도 합이 아니라 평균하므로 raw 최대값은 1 미만이다. Weight는 +0.2 이다.

현재 Foot_Link 원점은 약 8 mm 발 두께의 가운데에 있어 실제 sole보다 약 4 mm 높다. 따라서 이 h_i 는 정확한 collision-point clearance가 아니라 평지용 body-origin proxy다. 목표 0.054 m 는 원하는 sole clearance 0.050 m 에 이 origin offset 0.004 m 를 더한 값이다. $std=0.04$ m 기준에서 4 mm offset의 차이는 target 자체에 명시적으로 반영했으며, 평지에서는 environment origin이 지면 높이와 같으므로 별도 height scanner를 추가하지 않는다. 속도 gate는 높이 목표에 도달했더라도 거의 정지한 발이 점수를 받는 것을 막는다. 예를 들어 $v_{xy}=0.10$ m/s 에서는 $\tanh(0.2) \approx 0.197$, 0.20 m/s 에서는 $\tanh(0.4) \approx 0.380$, 0.50 m/s 에서는 $\tanh(1) \approx 0.762$ 이며 고속에서는 부드럽게 1에 포화된다. 따라서 이전 0.20 m/s scale보다 느린 swing Foot의 clearance shaping을 약하게 만든다. 이 설명은 선형 command와 pure-yaw의 접선 progress 절반에 적용된다. Pure-yaw의 나머지 절반은 제자리 회전 중 수직 lift 자체를 허용하기 위해 속도 gate 없이 높이 score를 사용한다.

no_jumps 는 $mdp.desired_contacts$ 를 $weight=-2.0$ 과 $force\ threshold=1.0$ N 으로 사용한다. 최근 contact-force history 에서 좌우 Foot 모두 threshold를 넘지 못한 경우에만 raw value 1 을 반환하므로, Reward Manager에서는 해당 양발 flight interval에 음의 점수가 적용된다. 한 발이라도 접촉 중이면 0이므로 정상 single stance와 toe-off는 직접 억제하지 않는다. 또한 이 term만으로 좌우 교대나 최대 single-stance 시간을 강제하지는 않는다. 두 velocity-tracking term의 최대 pre-dt 합 +2.0 을 flight 구간에서 상쇄하도록 -2.0 을 첫 실험값으로 사용한다.

$feet_swing_roll_l2$ 는 $weight=-1.0$ 로 활성화되어 swing 중 발바닥이 안쪽 또는 바깥쪽으로 말리는 현상을 억제한다. 각 Foot의 local +z 법선을 world frame으로 회전한 뒤 그 Foot 자체의 yaw frame으로 옮기고, lateral 성분 n_y^2 만 penalty로 반환한다. $feet_swing_pitch_l2$ 는 같은 벡터의 forward 성분 n_x^2 를 사용하며 현재 $weight=-0.1$ 로 toe-up과 toe-down 양쪽 기울임을 억제한다. Contact sensor의 $current_contact_time$ 이 0인 발에만 적용하고 swing 발 수로 평균하므로 지지 발은 두 항 모두 0이다. Pitch 가중치는 roll의 1/10이며 비활성 $feet_flat_orientation_l2$ 처럼 지지 발까지 world-up으로 당기지 않는다. 이는 soft penalty이므로 특정 각도나 heel-first 착지를 보장하지 않으며 필요한 swing 적응도 제한할 수 있다.

Contact sensor는 $update_period=0.002$ s 와 $history_length=self.decimation=5$ 를 사용한다. 따라서 no_jumps , $feet_slide$, $undesired_contacts$, $illegal_body_contact$ 처럼 $net_forces_w_history$ 를 검사하는 항목은 최근 policy interval의 5개 physics contact sample을 확인한다. 비활성 비교용 $feet_contact_force_l2$ 도 ground-filtered force_matri

x_w_history 를 같은 방식으로 검사한다. 반면 활성 FeetAirTimeTouchdownBiped class와 비교용 feet_air_time_touchdown_biped 함수는 모두 compute_first_contact(env.step_dt) 와 완료된 last_air_time 을 사용한다. 이 contact-force history는 policy observation history와 별개의 buffer이다.

현재 soft-landing 실험은 FeetTouchdownPitchL2 를 유지하면서 FeetTouchdownEdgeVelocityL2 의 직전 sample 비용만 5 배로 강화한다. 아래 FeetTouchdownVelocityL2 COM 항의 정의는 변경하지 않았다. Velocity class는 각 발 링크 COM의 이전 policy step world-XYZ 속도와 접촉 여부를 저장하고, 이전 sample이 공중이었던 발에 first contact가 발생할 때만 다음 raw penalty를 반환한다. 발별 값은 평균하지 않고 합산한다.

$$P_{landing} = \sum_i I_{first, i} I_{air, previous, i} [v_{x, previous, i}^2 + v_{y, previous, i}^2 + \max(0, -v_{z, previous, i})^2]$$

Reward Manager는 여기에 weight -10.0 과 dt=0.01 s 를 적용한다. 따라서 접근 중과 계속된 stance에는 직접 영향을 주지 않는다. Planar XY에는 dead zone을 두지 않고, Z는 아래 방향 성분만 사용한다. L2 특성상 작은 속도에는 작은 penalty만 생긴다. Reset 직후에는 이전 sample이 없으므로 초기 접촉을 제외한다.

위 식의 v 는 Foot 링크 자체의 COM c 속도다. 로봇 전체 COM 속도가 아니다. 현재 body_lin_vel_w 는 body_com_lin_vel_w 의 alias다. Isaac Lab의 위치와 속도 기본 alias는 기준점이 다르므로 이름의 body 만으로 판단하지 않는다.

```
body_pos_w      # body_link_pos_w: link-origin position
body_lin_vel_w  # body_com_lin_vel_w: link COM linear velocity
body_vel_w      # body_com_vel_w: COM linear velocity + angular velocity
```

2026-09-09 tdlink 실험에서는 body_link_lin_vel_w 를 사용했지만, 학습 부진 관찰 후 COM으로 복원했다. 원점 o 와 COM c 의 속도 관계는 다음과 같다. 모든 벡터는 world frame이다.

$$v_O^w = v_C^w + \omega_{foot}^w \times (p_O^w - p_C^w)$$

이 차이는 좌표계가 아니라 같은 world frame에서 평가하는 점의 차이다. 회전 중 두 속도는 다를 수 있다. 이번 복원은 원점 속도가 물리적으로 잘못되었다는 결론이 아니라 기존에 학습되던 reward 정의로 돌아가는 선택이다. 이전 10 ms sample, first-contact 조건, 발별 합산, 가중치와 나머지 학습 설정은 유지한다. 아래 두 TensorBoard tag도 다시 COM 속도를 집계한다. tdlink 로그는 같은 tag라도 원점 속도이므로 동일 지점의 측정처럼 직접 비교하거나 합치지 않는다.

- Metrics/feet_touchdown/mean_pre_touchdown_planar_speed [m/s]

- Metrics/feet_touchdown/mean_pre_touchdown_vertical_speed [m/s]

2026-09-14 착지 모서리 하강속도 실험

이 절의 계수 1 수식과 예시는 최초 실험을 보존한 것이다. 현재 코드는 아래 2026-09-17 절처럼 직전 sample 제공 비용에만 pre_touchdown_scale=5.0 을 곱한다. 좌표/점 선택/window 규칙은 동일하다.

목적은 COM이 천천히 움직여도 발 회전으로 toe/heel이 빠르게 내려오는 움직임을 약하게 억제하는 것이다. 이 항은 force penalty도, 발을 항상 수평으로 고정하는 orientation penalty도 아니다. 기존 COM 항 -10.0 을 유지하고 새 feet_touchdown_edge_velocity 만 weight=-0.1 로 추가한다. Recovery gate, 정지 접촉 -0.2, clearance, air-time, gain, DR, command/push 분포는 바꾸지 않는다.

Foot마다 toe 두 점과 heel 두 점의 world 속도를 계산한다. 실제 접촉점 API가 아니라 train collision box의 가상 모서리다. Toe x=0.175, heel x=-0.060, y=+/-0.045, z=0 [m]를 사용한다. 위치와 속도는 모두 같은 링크 원점 기준이며, 기존 진단과 새 보상이 동일한 kinematics helper를 사용한다.

$$v_{i,p}^w = v_{i,O}^w + \omega_i^w \times (R_i^w r_p), \quad s_i(k) = \max_p \max(0, -v_{i,p,z}^w(k)), \quad P_i(k) = s_i(k)^2$$

네 점의 값을 합산하지 않고 가장 빠른 하강점을 선택하므로, 두 toe 점이 같은 속도라고 벌점을 두 배로 만들지 않는다. 좌우 발의 비용은 합산한다. 가장 낮은 위치의 점을 선택하는 기존 진단과도 선택 기준이 다르다. 수평 또는 상승하는 병진운동만으로는 새 벌점이 생기지 않지만, 회전하여 어느 모서리가 내려오면 생긴다.

평가 시점

구간	처리
일반 swing	비용 0. 직전 policy sample의 최대 모서리 하강속도만 저장한다.
새 착지 감지	직전 sample이 공중이었고 유효한 history가 있을 때 P_i(k-1) 를 한 번 더한다.
첫 접촉부터 100 ms	현재 P_i(k) 를 10 ms마다 평가한다. 첫 접촉 sample도 포함하여 총 10회다.
이후 지지	새 항은 0. 다시 swing 후 새 착지하면 다음 window를 연다.

새 window 시작 mask를 B_i, 현재 window mask를 W_i 라고 하면:

$$r_{edge}(k) = -0.1 \Delta t \sum_i [B_i(k)P_i(k-1) + W_i(k)P_i(k)]$$

함수는 raw 합만 반환한다. Weight와 dt=0.01 s 는 Reward Manager가 한 번만 곱한다. 하강속도 0.4 m/s 가 직전 sample 과 window 10회 내내 동일한 한 발의 예라면 -0.1 * 0.01 * 0.4^2 * 11 = -0.00176 이다. 실제 평균 비용은 실제 속도와

착지 빈도에 따라 달라진다. 기존 event-only COM 항의 -10 과 가중치 숫자만 비교하지 않는다. -0.1 은 최초 실험값이며 최적값이 아니다.

Window는 발별 독립 정수 policy-step counter로 관리한다. 잠깐 contact가 끊겨도 남은 window는 유지하고, 그 안의 재접촉으로 timer를 연장하거나 직전 sample 비용을 다시 더하지 않는다. 환경 reset은 cache와 timer를 지우므로 초기 접촉에는 비용이 없다. Duration이 policy dt의 정수배가 아니면 다음 경계로 올림한다. 예를 들어 0.025 s 요청은 현재 dt에서 0.030 s 다. 100 ms는 정확히 10회다.

현재 reward는 policy sample만 보므로 sensor force history의 2 ms 속도 이력을 사용하는 것이 아니다. 짧은 충격을 놓칠 수 있으며, 일반 heel-to-toe 구름도 억제할 수 있다. 정지 명령 중 recovery 착지도 평가하지만 command/push 상태를 판정하여 보상을 풀어주는 recovery gate는 아니다.

추가 TensorBoard metric은 완료된 window마다 네 모서리 중 최대 하강속도의 시간 peak를 기록한다:

- Metrics/feet_touchdown/mean_peak_edge_downward_speed_0_20ms [m/s]
- Metrics/feet_touchdown/mean_peak_edge_downward_speed_20_100ms [m/s]

이 둘은 직전 공중 sample을 제외한 접촉 후 값이다. 제곱 비용이나 단일 전체 최대값이 아니며, 완료된 window의 peak를 환경 reset 때 event 수로 나눈 값이다. Reset으로 중단된 window는 이 집계에서 제외한다. 기존 COM/force/직전 toe/heel metric 의미는 바꾸지 않는다. 계수/dt 가 포함된 새 episode 비용은 Episode_Reward/feet_touchdown_edge_velocity 로 별도 기록된다.

직전 비교 run은 2026-09-10_19-45-38_concurrent_estimator_gain240_180_120_kd12_9_10_standcontact020_fresh25k/model_24999.pt 이다. 학습과 로그 검토는 완료했으나 이로부터 새로운 Sim2Sim/Sim2Real 검증을 추론하지 않는다. 모서리 run은 2026-09-14_16-41-13_concurrent_estimator_edgevel01_window100_fresh25k/model_24999.pt 까지 학습과 로그 검토를 완료했다. 사용자는 Isaac Sim Teleop에서 공중 heel-down/toe-down 회전 후 toe-first 착지를 보고했다. Sim2Sim/Sim2Real 검증은 보고되지 않았으며 기존 committed baseline을 교체하지 않는다. 동일 command/push에서 초기/후기 force, 모서리 속도, 정지, tracking, air-time을 비교한다. Force 파형은 300 ms까지 확인하여 충격을 window 밖으로 미룬 것은 아닌지 본다. 모서리 속도만 줄고 force가 남으면 체중 이동이나 접촉 모델 등 다른 원인도 고려하며 계수를 무조건 올리지 않는다.

2026-09-15 스윙 pitch 가중치 실험

당시 변경은 feet_swing_pitch_l2.weight: -0.1 -> -0.2 하나였다. 같은 상태에서 pitch 비용만 두 배이며 각도를 두 배로 제한한다는 뜻이 아니다. 기존 함수, COM 착지 -10.0, 모서리 하강 -0.1 / 100 ms, roll -1.0, 정지 접촉 -0.2, gain, clearance, air-time, DR, command/push, Actor/ONNX는 유지한다. Force는 여전히 기록 전용이며 recovery gate나 새 접촉 조건을 추가하지 않는다.

직전 모서리 run과 9월 10일 gain run의 마지막 500 iteration 평균을 비교하면 착지창 force peak는 3368 -> 2712 N, 후기 20-100 ms Fz peak는 3158 -> 2508 N 로 감소했지만, 스윙 pitch 비용 절댓값은 0.000523 -> 0.002173 으로 약 4.15배 커졌다. 이는 각도가 4.15배라는 뜻이 아니다. 사용자의 toe-first 관찰과 일치하는 가설은 접촉 후 회전이 공중으로 앞당겨졌다는 것이지만, 시간별 pitch/contact 자료 없이 원인으로 확정하지 않는다. 모서리 속도 비용에는 착지 자세 목표가 없다.

이번 실험은 기존 swing-only pitch 제약을 강화하여 공중에서 과도한 toe-down 회전을 줄이려는 것이다. 지지 발의 구름을 직접 제한하거나 heel strike를 강제하는 방식은 아니다. 2026-09-15_10-32-59_concurrent_estimator_edgevel01_window100_swingpitch02_fresh25k/model_24999.pt 까지 학습과 로그 검토를 완료했다. 직전 run 대비 마지막 500 iteration force peak 평균은 2712 -> 2236 N 이지만 toe 직전 하강속도는 0.418 -> 0.464 m/s 로 증가했다. 가중치를 제거한 pitch 비용은 약 72% 감소했으나, 사용자는 Isaac Sim Teleop에서 더 심한 공중 toe-down과 새 정지 발 세움 현상을 보고했다. Sim2Sim/Sim2Real 검증은 보고되지 않았으며 이 실험을 새 baseline으로 채택하지 않았다.

2026-09-16 스윙 pitch 복원

사용자 승인으로 pitch weight만 -0.2 -> -0.1 로 복원했다. COM/edge 보상, gain 및 나머지 설정은 유지한다. 새 벌점, recovery gate, 기록 기능은 추가하지 않았다. 이미 학습된 -0.2 정책의 action은 설정 복원만으로 바뀌지 않으므로 이전 동작 비교에는 9월 14일 checkpoint를 사용한다. 새 학습은 실행하지 않았다.

복원 당시 제안한 진단 우선 계획은 이후 철회했고, 아래 착지 자세 항을 사용자 승인으로 추가했다. 새 recorder는 구현하지 않았다. Contact flag는 Foot body의 접촉 여부이며 발바닥 전체 접촉 판정이 아니다. Toe만 닿아도 contact로 판단되면 기존 swing pitch는 꺼진다. 이 시점의 자세를 별도 평가하는 것이 새 항의 목적이다.

2026-09-16 착지 pitch 추가

feet_touchdown_pitch_l2 = RewTerm(func=mdp.FeetTouchdownPitchL2, weight=-1.0) 을 추가했다. 기존 feet_swing_pitch_l2=-0.1 을 대체하지 않는다. COM 착지 속도 -10.0, 모서리 속도 -0.1 과 100 ms window, 나머지 reward/gain/DR/command/push/Actor/Critic/ONNX는 변경하지 않는다. 2026-09-16_12-19-45_concurrent_estimator_edgevel01_window100_tdpitch1_fresh25k/model_24999.pt 까지 학습과 로그 검토를 완료했다. 사용자는 Isaac Sim Teleop에서 안정적인 standing과 전진 heel strike, toe 착지 해소를 보고했다. 다만 첫 충격은 크고 착지 후 충격은 작다고 관찰했다. Sim2Sim/Sim2Real 검증은 보고되지 않았으며 committed baseline은 그대로 유지한다.

발의 local +Z 법선을 world로 회전하고 그 발의 yaw를 제거한다. 기존 swing pitch와 동일한 식이다. e_i 는 각도 [rad] 자체가 아니라 무차원 벡터 성분의 제곱이다.

$$n_i^{yaw}(k) = R_z(-yaw_i(k))R_i(k)[0, 0, 1]^T$$

$$e_i(k) = (n_{i,x}^{yaw}(k))^2$$

ZYX roll/pitch/yaw 기준으로 $e = \sin(\text{pitch})^2 * \cos(\text{roll})^2$ 이므로 roll이 작을 때 $\sin(\text{pitch})^2$ 에 가깝다. Toe-up/down 부호에 무관하며 heel-first를 지정하지 않는다.

환경/발별로 이전 오차, 이전 접촉 flag, 이전 sample 유효 여부를 저장한다. I_{td} 는 first_contact AND has_previous_sample AND NOT previous_in_contact 다. 첫 접촉 순간에는 다음 raw cost를 발별로 합산하고 Reward Manager가 weight와 policy dt를 한 번 적용한다.

$$P_{td, pitch}(k) = \sum_i I_{td,i}(k) \max(e_i(k-1), e_i(k))$$

$$r_{td, pitch}(k) = -1.0 \times 0.01 \times P_{td, pitch}(k)$$

```
ordinary swing -> save current error/contact; new pitch cost = 0
first contact  -> max(previous error, current error); charge once
continued stance -> new pitch cost = 0
toe-off        -> new pitch cost = 0
next landing   -> evaluate a new observed airborne-to-contact transition
reset          -> clear selected histories; skip initial contact sample
```

이 항에는 command/recovery gate, 각도 deadband, 착지 후 자세 유지 window가 없다. 정지 중 recovery 착지도 같은 방식으로 평가한다. 양발이 동시에 착지하면 두 비용을 합산하며 공중 발 수로 나누지 않는다. 접촉이 끊어진 상태가 policy sample로 관측된 뒤 다시 착지하면 새 사건이다. 별도의 bounce debounce는 없다.

이전/현재 sample은 policy dt 10 ms 간격이며 정확한 충돌 직전/직후 physics sample을 뜻하지 않는다. Contact sensor의 2 ms x 5 force history에서 pitch 최대값을 찾는 것도 아니다. 현재 자세는 충돌 후 변화를 이미 포함할 수 있으므로 두 sample 중 큰 값을 사용해 현재 sample만의 빠른 정렬로 오차가 사라지는 것을 줄인다. 10 ms 사이에 발생한 더 큰 회전은 놓칠 수 있다.

가중치 -1.0 은 event-only 시작 실험값이지 검증된 최적값이 아니다. Roll이 0이고 두 sample 중 큰 pitch가 20도이면 raw cost는 약 0.117, 한 발 착지의 weighted step reward는 약 -0.00117 이다. 작은 heel-first에도 작은 벌점, 큰 toe-up/down에는 큰 벌점이 생긴다. 평평한 착지, 낮은 GRF 또는 양발 전체 면접촉을 보장하지 않으며 stance toe-off에 이 항을 계속 적용하지도 않는다. 이 실험은 평지 world-horizontal 기준이다.

TensorBoard의 Episode_Reward/feet_touchdown_pitch_l2 는 Reward Manager가 자동으로 기록하는 weighted reward 합 / 최대 episode 시간이다. 실제 pitch 각도나 착지당 평균 오차가 아니다. 추가 custom metric이나 매 step GPU-to-CPU 집계는 넣지 않았다. 기존 toe/heel/force 진단은 유지한다.

2026-09-17 착지 직전 모서리 비용 강화

착지 pitch의 관찰된 개선은 유지하고 첫 충격을 줄이기 위해, 기존 모서리 항에 pre_touchdown_scale=5.0 하나만 지정한다. 함수 기본값은 1.0 이므로 이 옵션을 생략한 이전 설정은 기존 계산과 같다. 0 은 직전 비용 없이 접촉 후 window만 평가하며, 음수/NaN/무한대는 거부한다. Gain, COM 착지 -10, 착지 pitch -1, swing pitch -0.1, force 기록 전용 설정과 나머지 학습 조건은 유지한다.

s_i 는 네 모서리 중 최대 하강속도, B_i 는 새 착지, w_i 는 활성 window mask다. 현재 전체 식은 다음과 같다.

$$r_{edge}(k) = -0.1 \Delta t \sum_i [5B_i(k)s_i(k-1)^2 + W_i(k)s_i(k)^2]$$

```
weight = -0.1
landing_window_s = 0.10
pre_touchdown_scale = 5.0
# Raw cost: scale AFTER squaring, not (5 * speed)**2.
penalty = (5.0 * previous_speed.square() * new_landing
           + speed.square() * active).sum(dim=1)
```

직전 비용의 실효 계수는 -0.5, 현재 window 비용은 -0.1 이다. 둘 다 Reward Manager가 dt=0.01 s 를 한 번 곱한다. 예를 들어 한 발의 직전 하강속도가 0.4 m/s 이면 해당 비용만 -0.00016 -> -0.0008 이다. 같은 착지에서 현재 속도가 0.1 m/s 이면 현재 sample 비용 -0.00001 은 그대로다. 속도 자체를 5배로 만들어 25배 비용을 주는 것이 아니다.

이전 값은 첫 접촉을 감지하기 한 policy step 전에 저장한 값이다. 2 ms physics 기준의 정확한 충돌 직전 속도나 force history의 최대 속도는 아니다. 네 모서리 XYZ 속도를 계산하지만 비용에는 world-Z 하강 성분의 최댓값만 사용한다. 좌우 발 비용은 합산하고 네 모서리 비용은 합산하지 않는다. 재접촉 중복 방지, 발별 100 ms window와 reset 처리는 유지한다. 물리 속도 metric에 5를 곱하지 않으며 새 metric이나 CPU 집계를 추가하지 않는다.

비교 run은 위 9월 16일 tdpitch1 의 model_24999.pt 다. 마지막 500 iteration 평균에서 초기 0-20 ms Fz peak는 약 1922 N, 후기 20-100 ms 는 약 1446 N 이다. 9월 14일 모서리-only run의 같은 값은 약 766 / 2508 N 이었다. 이는 전역 최대 force가 아니라 window별 peak 평균이다. 개선된 후기 구간까지 일괄 강화하지 않고 직전 비용만 조절하는 이유다. 계수 5는 실험값이며 충격 감소를 보장하지 않는다. 이 설정으로 fresh 25,000 iteration 학습을 마쳤고 9월 18일 사용자 Teleop 관찰을

바탕으로 현재 개발 baseline으로 보존했다. 코드 snapshot은 903318c 다. 학습 당시에는 2c6fe26 기반 미커밋 코드를 사용했고, snapshot은 학습 완료 후 저장 설정과 대조하여 만들었다. 이 기록은 새 Sim2Sim/Sim2Real 검증을 의미하지 않는다.

마지막 500 iteration 평균 비교 (2026-09-18)

항목	9월 16일 pre1	9월 17일 pre5
착지창 접촉 합력 크기 peak [N]	2129	1966
초기 0-20 ms Fz peak [N]	1922	1693
후기 20-100 ms Fz peak [N]	1446	1494
직전 heel 하강속도 [m/s]	0.456	0.363
보상 대상 touchdown 평균 air-time [s]	0.367	0.355
XY 추종 오차 지표	0.1469	0.1409

모든 force는 착지 사건 peak의 집계 평균이며 전역 최대값이 아니다. 첫 충격과 heel 하강속도는 감소했지만 후기 force는 약 3.3% 증가했고 air-time은 조금 짧아졌다. Standing contact 벌점 절댓값은 약 10.6% 감소, no-jumps 벌점 절댓값은 약 21.4% 증가했다. 각도/발걸음 빈도나 드문 충격을 이 값만으로 확정하지 않는다. 사용자의 부드러운 착지 관찰은 Isaac Sim keyboard Teleop 단계의 결과로만 기록한다.

이전 COM 비교 기록

COM 기준 비교 정책은 2026-09-08_16-59-33_concurrent_estimator_hip020_standcontact020_fresh25k/model_24999.pt 이다. 사용자는 Isaac Sim에서 정지 stepping 해소와 보행 유지를 확인했다. 원점 기준 trial은 2026-09-09_15-26-01_concurrent_estimator_hip020_standcontact020_tdlink_fresh25k이며 중간 학습이 부진했다. COM 복원 run은 2026-09-09_17-36-08_concurrent_estimator_hip020_standcontact020_tdcom_restore_fresh25k이며 model_24999.pt 까지 fresh 학습을 완료했다. 사용자는 Isaac Sim Teleop에서 이전과 비슷한 보행/정지와 잔여 착지 후 충격, 전방 recovery toe 걸림을 보고했다. MuJoCo Sim2Sim/Sim2Real 검증은 보고되지 않았다. 이 run은 이전 240/12, 160/8, 80/8 actuator gain을 사용한다. 2026-09-10 새 gain 실험은 240/12, 180/9, 120/10이며 reward 함수/가중치와 4 ms delay, DR, observation, command는 유지한다. 새 gain의 학습과 검증은 별도이며 기존 checkpoint가 자동으로 새 제어 조건에 검증된 것은 아니다. 이전 실기 검증 기준은 2026-08-12_23-45-39_privileged250_gain240_160_80_air050_w2_tdvel10_ar01_ar2_005_noforce_delay4ms_fresh20k 의 model_19999.pt 다. 이 정책은 Z 전용 penalty로 학습되었으며, 마지막 checkpoint에서 평균 착지 직전 하강속도는 약 0.044 m/s 였고 velocity tracking과 평균 완료 air time은 유지되었다. 학습 checkpoint는 Isaac Lab log directory에 보존하며 Git에는 포함하지 않는다.

FeetContactForceL2 class의 shaping term은 feet_contact_force=None 이다. 별도 feet_contact_force_metrics term 은 metric_only=True 와 weight 1.0 으로 매 step 실행되지만 함수가 항상 0을 반환하므로 total reward와 policy gradient에 영향을 주지 않는다. 이 logging-only term은 touchdown 후 0.10 s 동안의 peak GRF를 집계해 TensorBoard에 기록한다.

FeetTouchdownDiagnostics 도 항상 0을 반환하는 측정 전용 term이다. RoK4 collision sole box의 네 모서리를 foot body frame에서 다음과 같이 정의한다.

$$r_{toe, \pm} = [0.175, \pm 0.045, 0]^T, r_{heel, \pm} = [-0.060, \pm 0.045, 0]^T$$

각 모서리의 world-frame 위치와 속도는 다음 rigid-body kinematics로 계산한다. 0 는 Foot 링크 원점, c 는 그 발 링크 자체의 COM이다. Local sole offset은 회전 행렬로 world frame으로 변환한다.

$$p_{edge}^w = p_O^w + R_{foot}^w r_{edge}, v_{edge}^w = v_O^w + \omega_{foot}^w \times (R_{foot}^w r_{edge})$$

동일한 계산을 COM 기준으로 쓰면 다음과 같다.

$$v_{edge}^w = v_C^w + \omega_{foot}^w \times (p_{edge}^w - p_C^w)$$

2026-09-09 수정 전 진단은 v_C 에 원점 기준 lever arm을 더하는 오류가 있었다. 수정 후에는 body_link_lin_vel_w 로 v_0 를 받아 위치와 속도의 기준점을 일치시킨다. 기존 toe/heel/최저 모서리 속도 로그는 잘못된 기준으로 계산한 값이며, 원래 per-sample 각속도/자세/COM offset이 없으면 집계값만으로 복원할 수 없다. 기존 checkpoint를 재학습 없이 다시 평가하되 이전 곡선과 수정 후 곡선을 직접 합치지 않는다. 이 진단 수정 자체는 기록 전용 term에 한정되었다. 그 뒤 착지 reward를 원점으로 바꾸는 별도 실험을 거쳐 현재 reward/착지 속도 metric은 다시 COM 기준으로 복원했다. Toe/heel 진단은 원점 속도와 원점 기준 lever arm을 사용하는 수정된 계산을 유지한다. Force 집계 방식, Actor/ONNX 계약, 기존 checkpoint도 유지한다.

따라서 발 원점 속도가 작더라도 pitch/roll 각속도로 toe 또는 heel이 빠르게 내려오는 foot slap을 측정할 수 있다. Toe 두 모서리 중 낮은 점, heel 두 모서리 중 낮은 점, 네 모서리 전체 중 가장 낮은 점을 각각 선택한다. First contact가 발생하면 직전 policy sample의 절대 X/Y 속도, 하강속도, 최저 모서리 planar speed를 사건당 한 번 누적한다.

동시에 filtered contact-force history의 world-Z 성분만 사용하여 first contact 이후 0-20 ms 초기 충돌과 20-100 ms 체중 인수 구간의 peak normal force를 따로 누적한다. 기존 mean_peak_normal_force 는 호환성을 위해 이름을 유지한 100 ms 합력 크기 $\|F\|$ 이고, 새 두 force metric은 실제 $\max(F_z, 0)$ 이므로 서로 같은 물리량이 아니다. 이 진단은 observation, action, network, gain, DR 또는 reward gradient를 변경하지 않으며 기존 checkpoint를 Play/Teleop으로 실행해도 사용할 수 있다. 학습에서는 TensorBoard에 기록되고, Teleop에서는 episode 종료 또는 keyboard R reset 직후 같은 물리 단위 평균이 터미널에 출력된다.

Air-time과 touchdown velocity class는 episode 누적값과 event count를 GPU tensor로 유지하고, 환경 reset 때 Metrics/feet_touchdown/* 의 event-weighted 평균을 기록한다. 기존 Gym 호환 stateless feet_contact_velocity_l2 와 feet_contact_force_l2 함수는 비교용으로 남아 있지만 현재 Reward Manager term에는 연결하지 않는다.

Play 환경은 현재 standing 검증을 위해 `lin_vel_x=0.0 m/s` 를 사용하고 lateral/yaw command도 0으로 고정한다. 별도의 Teleop 환경은 긴 resampling interval을 사용하고, gamepad 또는 keyboard의 `[lin_vel_x, lin_vel_y, ang_vel_z]` 를 동일한 base-frame command buffer에 직접 기록한다. 입력은 학습 범위 $(-0.3, 0.85)$ m/s, $(-0.3, 0.3)$ m/s, $(-0.6, 0.6)$ rad/s 안으로 scale된다. 이는 inference command source만 바꾸며 reward 정의나 학습 checkpoint 자체를 변경하지 않는다. Play와 Teleop에서는 학습 전용 periodic freeze를 비활성화한다.

Termination과 Reward의 연결

RoK4 flat task는 Isaac Lab의 `TerminationsCfg` 를 수정하지 않고 로컬 `RoK4TerminationsCfg` 로 상속한다. 부모의 `time_out` 은 유지하고 `base_contact` 는 비활성화한 뒤 `illegal_body_contact` 를 추가한다.

```
class RoK4TerminationsCfg(TerminationsCfg):
    base_contact = None
    illegal_body_contact = DoneTerm(
        func=mdp.illegal_contact,
        params={
            "sensor_cfg": SceneEntityCfg(
                "contact_forces",
                body_names=R0K4_ILLEGAL_CONTACT_BODY_NAMES,
            ),
            "threshold": 1.0,
        },
    )
```

`R0K4_ILLEGAL_CONTACT_BODY_NAMES` 에는 `Base_Link`, `Upper_Body_Link` 와 좌우 `Hip_Yaw`, `Hip_Roll`, `Thigh`, `Calf`, `Ankle_Pitch`, `Ankle_Roll` link가 포함된다. 정상 접촉 body인 `L_Foot_Link` 와 `R_Foot_Link` 만 제외된다. 선택된 body 중 하나라도 contact force가 1.0 N 을 넘으면 episode가 종료된다.

`mdp.is_terminated` 는 non-timeout termination을 감지하므로 `illegal_body_contact` 가 발생하면 `termination_penalty = -200.0` 가 함께 작동한다. 반면 부모에서 상속한 정상 `time_out` 에는 이 penalty가 적용되지 않는다. 여러 body가 동시에 접촉해도 termination flag는 boolean이므로 termination penalty는 한 번만 적용된다. Contact sensor는 상대 물체 종류를 구분하지 않은 net contact force를 사용하므로 self-collision으로 선택 body에 큰 접촉력이 생겨도 종료될 수 있다.

Touchdown air-time 로그 해석

Directional-gait 기준 run 2026-08-03_14-58-46_touchdown_air_symmetric_x_fastforward_fresh20k 의 `Episode_Reward/feet_air_time` 은 다음처럼 변했다.

Touchdown feet-air-time checkpoint 값

checkpoint	logged reward
model_9999.pt	0.00278
model_14999.pt	0.00244
model_19999.pt	0.00297
마지막 500 iteration 평균	0.00272

이 scalar는 발의 평균 air-time 초 단위 값이 아니다. Reward Manager가 기록하는 episode-normalized 값에는 policy dt=0.01 s, reward weight=2.0, 유효 touchdown 빈도, 완료된 air-time, planar moving-command mask가 함께 들어간다. 개념적으로 다음 곱에 가깝다.

$$\bar{r}_{air} \approx 0.01 \cdot 2.0 \cdot f_{touchdown} \cdot \mathbb{E}[T_{air} - 0.50] \cdot p_{moving}$$

규칙적인 교대 보행에서는 touchdown 빈도와 step air-time이 서로 반비례할 수 있으므로 이 scalar 하나로 τ_{air} 을 역산할 수 없다. 또한 10k, 15k, 20k 값이 단조 증가하지 않아도 보행이 악화되었다는 뜻이 아니다. 현재 구현은 정확한 평균 비교를 위해 touchdown last_air_time 과 착지 직전 planar/vertical 속도를 event count로 나눈 물리 단위 TensorBoard metric을 함께 기록한다. 중앙값/상위 백분위와 초당 touchdown 횟수는 아직 집계하지 않는다.

- Metrics/feet_touchdown/mean_pre_touchdown_planar_speed [m/s]
- Metrics/feet_touchdown/mean_pre_touchdown_vertical_speed [m/s]
- Metrics/feet_touchdown/mean_air_time [s]
- Metrics/feet_touchdown/mean_peak_normal_force [N]
- Metrics/feet_touchdown/mean_pre_touchdown_toe_abs_vx [m/s]
- Metrics/feet_touchdown/mean_pre_touchdown_toe_abs_vy [m/s]
- Metrics/feet_touchdown/mean_pre_touchdown_toe_downward_speed [m/s]
- Metrics/feet_touchdown/mean_pre_touchdown_heel_abs_vx [m/s]
- Metrics/feet_touchdown/mean_pre_touchdown_heel_abs_vy [m/s]
- Metrics/feet_touchdown/mean_pre_touchdown_heel_downward_speed [m/s]
- Metrics/feet_touchdown/mean_pre_touchdown_lower_edge_planar_speed [m/s]
- Metrics/feet_touchdown/mean_pre_touchdown_lower_edge_downward_speed [m/s]
- Metrics/feet_touchdown/mean_peak_normal_force_0_20ms [N]
- Metrics/feet_touchdown/mean_peak_normal_force_20_100ms [N]

mean_peak_normal_force 는 기존 TensorBoard tag 호환을 위해 이름을 유지한다. 실제 집계값은 filtered world-frame 접촉 합력 $||[F_x, F_y, F_z]||$ 의 landing-window peak다. feet_contact_force_metrics 는 이 값만 기록하며 reward에는 0을 반환한다.

현재 설계 의도

현재 reward는 다음 목표를 동시에 가진다.

- 1. command velocity를 따라간다.
- 2. upright 자세를 유지한다.
- 3. 한 발씩 드는 biped stepping pattern을 만든다.
- 4. 양발이 동시에 공중에 뜨는 flight phase를 줄이되 정상 single stance는 허용한다.
- 5. standing과 moving 모두에서 좌우 Foot의 signed lateral separation 하한만 두어 다리 교차를 억제한다.
- 6. foot-flat orientation과 목표 stance-width 함수는 구현되어 있지만 현재 비활성이다.
- 7. 발 미끄러짐을 줄인다.
- 8. torque, joint acceleration, 강화된 action rate와 second action rate로 움직임과 고주파 진동을 줄인다.
- 9. ankle limit, hip/torso deviation을 제한하며 hip yaw/roll은 ``-0.2``, hip pitch는 ``-0.01`` 을 사용한다.
- 10. zero command에서 dense velocity/upright/base-height/smoothness reward로 정지 안정성을 유도하며, 현재 ablation은 full-body standing default-pose penalty를 사용하지 않는다.
- 11. Foot를 제외한 body 접촉을 실패 종료로 처리한다.

튜닝 시 우선 확인할 항목

현상	먼저 볼 항목
너무 자주 넘어진다	termination_penalty, illegal_body_contact, self-collision, flat_orientation_l2, 초기 자세
거의 걸지 않고 버틴다	track_lin_vel_xy_exp, command range, feet_air_time
발이 많이 미끄러진다	feet_slide, foot collision, friction, contact sensor
양발이 동시에 공중에 뜨는 점프가 반복된다	no_jumps, contact-force threshold/history, feet_air_time 의 single-stance 보상 크기
발날 또는 발끝으로 착지한다	ankle actuator gain/target, Foot body frame, contact sensor를 먼저 확인한다. feet_flat_orientation_l2 는 world-up 기준 함수이므로 평지 진단 실험에서만 선택적으로 활성화한다.
다리를 과도하게 벌리거나 교차한다	feet_lateral_separation_l2 의 signed width와 penalty를 먼저 확인한다. 이 term은 교차/최소 폭만 다루며 넓은 stance는 제한하지 않는다. 넓은 stance 문제에는 joint_deviation_hip 와 비활성 feet_stance_width_l2 를 별도로 검토한다.
관절이 떨린다	action_rate_l2, second_action_rate_l2, actuator_acc_l2, actuator PD gain, action scale
토크가 과도하다	actuator_torques_l2, actuator torque limit, action scale
관절이 limit 근처로 간다	dof_pos_limits, joint_action_target_pos_limits, 해당 joint의 default pose, action scale, USD hard limit